



DESIGN DOCUMENT

# EnerTrack

*Database, RLS, API & Directory Design*

---

Intern Pre-Assignment — Phase 2: Design (revised after Review 2)

Prepared for **AiLaB, Inc.**

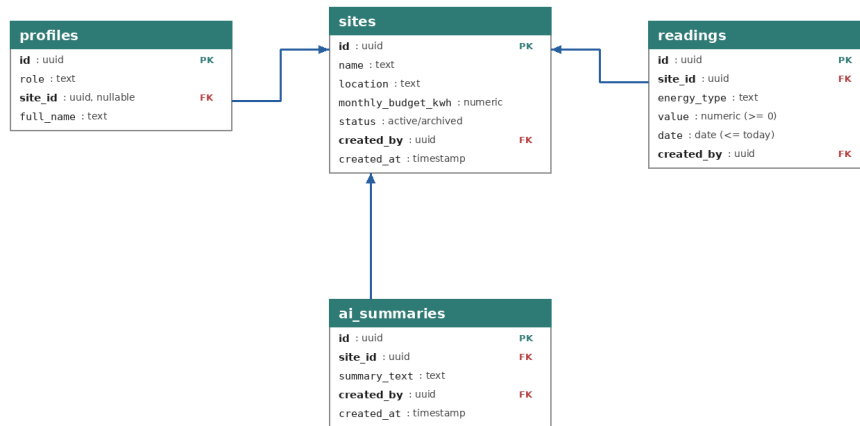
Prepared by **MESTARI Ayoub**

September 2026

# 1. Database Design

This document covers the five Phase 2 deliverables: (1) database design, (2) role & permission design, (3) RLS policy design, (4) API design, and (5) directory structure — building directly on the roles and data model introduced in the Phase 1 Requirements Document. Section 6 lists the decisions taken after Review 2.

Four tables cover the MVP scope. All primary keys are UUIDs (Supabase default).



PK = Primary Key FK = Foreign Key → references  
Note: created\_by columns in sites, readings, and ai\_summaries also reference profiles.id (omitted above for clarity).

## profiles

Extends auth.users; the row is created automatically at sign-up (Section 3.4).

| Column    | Type                           | Notes                                                   |
|-----------|--------------------------------|---------------------------------------------------------|
| id        | uuid (PK)                      | References auth.users.id                                |
| role      | text                           | CHECK in (site_manager, energy_manager, direction)      |
| site_id   | uuid (FK → sites.id), nullable | Null until Direction assigns a site (Site Manager only) |
| full_name | text                           | Display name                                            |

Constraint enforcing “1 site = 1 Site Manager” at database level (partial unique index):

```

CREATE UNIQUE INDEX one_manager_per_site
ON profiles (site_id)
WHERE role = 'site_manager' AND site_id IS NOT NULL;

```

## sites

| Column             | Type      | Notes                                                                                                                        |
|--------------------|-----------|------------------------------------------------------------------------------------------------------------------------------|
| id                 | uuid (PK) |                                                                                                                              |
| name               | text      |                                                                                                                              |
| location           | text      |                                                                                                                              |
| monthly_budget_kwh | numeric   | Electricity budget in kWh, the comparison target for anomaly detection. Budgets for other energy types are out of MVP scope. |

| Column     | Type                    | Notes                                                                      |
|------------|-------------------------|----------------------------------------------------------------------------|
| status     | text                    | active (default) / archived. Sites are archived, never physically deleted. |
| created_by | uuid (FK → profiles.id) | Must be a Direction account                                                |
| created_at | timestamp               | Default: now()                                                             |

## readings

| Column      | Type                    | Notes                                                          |
|-------------|-------------------------|----------------------------------------------------------------|
| id          | uuid (PK)               |                                                                |
| site_id     | uuid (FK → sites.id)    | ON DELETE RESTRICT (sites are archived, not deleted)           |
| energy_type | text                    | electricity / gas / fuel / water                               |
| value       | numeric                 | CHECK (value >= 0). Always stored in the canonical unit below. |
| date        | date                    | CHECK (date <= current_date)                                   |
| created_by  | uuid (FK → profiles.id) | Equivalent to the site's single Site Manager in this MVP       |

Additional constraints on readings:

```

UNIQUE (site_id, energy_type, date)  -- no duplicate reading per day and type
CREATE INDEX readings_site_date_idx ON readings (site_id, date);

```

## Units (decision)

No free-text unit column: each energy type has one fixed unit, shown in the entry form, so comparisons and the numbers passed to the LLM stay consistent.

| energy_type | Canonical unit |
|-------------|----------------|
| electricity | kWh            |
| gas         | m <sup>3</sup> |
| fuel        | litres (L)     |
| water       | m <sup>3</sup> |

## ai\_summaries

Stores generated explanations for later review and export.

| Column       | Type                    | Notes                        |
|--------------|-------------------------|------------------------------|
| id           | uuid (PK)               |                              |
| site_id      | uuid (FK → sites.id)    |                              |
| summary_text | text                    | The AI-generated explanation |
| created_by   | uuid (FK → profiles.id) | Who triggered the generation |
| created_at   | timestamp               | Default: now()               |

## 2. Role & Permission Design

Which role can Read / Create / Update / Delete rows in each table. “Own site” means restricted to the row(s) matching the user's assigned site\_id.

### sites

| Role           | Read      | Create | Update              | Delete               |
|----------------|-----------|--------|---------------------|----------------------|
| Site Manager   | Own site  | No     | No                  | No                   |
| Energy Manager | All sites | No     | No                  | No                   |
| Direction      | All sites | Yes    | Yes (incl. archive) | No (archive instead) |

### readings

| Role           | Read      | Create   | Update   | Delete   |
|----------------|-----------|----------|----------|----------|
| Site Manager   | Own site  | Own site | Own site | Own site |
| Energy Manager | All sites | No       | No       | No       |
| Direction      | All sites | Yes      | Yes      | Yes      |

### profiles

| Role           | Read                                          | Create                    | Update                       | Delete |
|----------------|-----------------------------------------------|---------------------------|------------------------------|--------|
| Site Manager   | Own row                                       | No (automatic at sign-up) | Own row, full_name only      | No     |
| Energy Manager | All rows (read-only, to show reading authors) | No                        | Own row, full_name only      | No     |
| Direction      | All rows                                      | Yes (via invite)          | All rows (incl. role / site) | Yes    |

### ai\_summaries

| Role           | Read      | Create    | Update | Delete |
|----------------|-----------|-----------|--------|--------|
| Site Manager   | Own site  | Own site  | No     | No     |
| Energy Manager | All sites | All sites | No     | No     |
| Direction      | All sites | Yes       | No     | Yes    |

## 3. RLS Policy Design

Row Level Security policies enforce access directly in PostgreSQL, so access control holds even if a client bypasses the UI. RLS decides which rows a user can touch; it cannot, on its own, decide which columns of an allowed row can be changed — that limit is handled in Section 3.3.

### 3.1 Single source of truth for role and site

Policies, triggers and middleware all read the current user's role and site from the profiles table, through two helper functions. They are SECURITY DEFINER, so they read profiles without triggering RLS; this avoids the “infinite recursion detected in policy for relation profiles” error that appears when a profiles policy queries profiles.

```
CREATE FUNCTION get_my_role() RETURNS text
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
  SELECT role FROM profiles WHERE id = auth.uid();
$$;

CREATE FUNCTION get_my_site_id() RETURNS uuid
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
  SELECT site_id FROM profiles WHERE id = auth.uid();
$$;
```

Supabase session JWTs do not contain role or site\_id by default. The alternative (custom JWT claims through an auth hook) is deliberately not used in the MVP; middleware instead runs one small server-side query per request against the same profiles table, so all three layers (policies, triggers, middleware) share the same source.

### 3.2 Policies per table

#### sites

- SELECT: `get_my_role() IN ('energy_manager', 'direction') OR id = get_my_site_id()`.
- INSERT / UPDATE: `get_my_role() = 'direction'`. No DELETE policy (sites are archived through UPDATE on status).

#### readings

- SELECT: `get_my_role() IN ('energy_manager', 'direction') OR site_id = get_my_site_id()`.
- INSERT / UPDATE / DELETE: `get_my_role() = 'direction' OR (get_my_role() = 'site_manager' AND site_id = get_my_site_id())`. Energy Manager has no write access.

#### profiles

- SELECT: `id = auth.uid() OR get_my_role() IN ('energy_manager', 'direction')`.
- UPDATE: `id = auth.uid() OR get_my_role() = 'direction'` (which columns may change is enforced by the trigger in 3.3).
- INSERT: no client-side policy; rows are created by the sign-up trigger (3.4) or by the server-side invite route.
- DELETE: `get_my_role() = 'direction'`.

#### ai\_summaries

- SELECT: `get_my_role() IN ('energy_manager', 'direction') OR site_id = get_my_site_id()`.
- INSERT: same condition as SELECT (site\_manager limited to own site). DELETE: direction only.

When a Site Manager's `site_id` is null (pending assignment), the condition `site_id = get_my_site_id()` evaluates to NULL and matches no rows — the empty/pending state defined in the requirements document, enforced at the database level and not only in the UI.

### 3.3 What RLS cannot do: protecting profiles.role

RLS works at row level. A policy on profiles that lets users update their own row would also let a Site Manager run `UPDATE profiles SET role = 'direction'` on that row, because the row is theirs — a privilege escalation. A `WITH CHECK` clause only sees the row after the update, so it cannot express “role must not change”. The protection therefore lives outside RLS, in a trigger that compares the old and new row:

```
CREATE FUNCTION protect_profile_columns() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF auth.uid() IS NOT NULL
        AND get_my_role() <> 'direction'
        AND (NEW.role IS DISTINCT FROM OLD.role
            OR NEW.site_id IS DISTINCT FROM OLD.site_id) THEN
        RAISE EXCEPTION 'Only direction can change role or site_id';
    END IF;
    RETURN NEW;
END; $$;

CREATE TRIGGER protect_profile_columns
BEFORE UPDATE ON profiles
FOR EACH ROW EXECUTE FUNCTION protect_profile_columns();
```

`auth.uid()` is null for server-side calls made with the service-role key and for the SQL editor (used for the initial seed and the invite route), so those are not blocked; every signed-in non-Direction user is. Column-level privileges (`REVOKE UPDATE`, then `GRANT UPDATE (full_name)`) are an equivalent alternative.

#### Attacker test to run in Phase 3

Signed in as a site\_manager, run `UPDATE profiles SET role = 'direction' WHERE id = auth.uid();` (and the same for `site_id`). Expected result: the statement fails with the exception above. The same test is repeated through the API route.

### 3.4 Profile creation at sign-up

Public sign-up is the primary path. A trigger on `auth.users` creates the matching profiles row with the default role and no site, so a new user is never left without a role:

```
CREATE FUNCTION handle_new_user() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    INSERT INTO profiles (id, role, site_id, full_name)
    VALUES (NEW.id, 'site_manager', NULL,
        NEW.raw_user_meta_data ->> 'full_name');
    RETURN NEW;
END; $$;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION handle_new_user();
```

Invitation (`/api/users/invite`) is a secondary path: Direction uses it to create Energy Manager or Direction accounts, since public sign-up always produces a site\_manager. Mechanically, the invited user first receives the default site\_manager profile from the sign-up trigger; the route handler then sets the intended role with

the service-role key, and the protect\_profile\_columns trigger lets this through because auth.uid() is null for service-role calls. The first Direction account is seeded manually in Supabase.

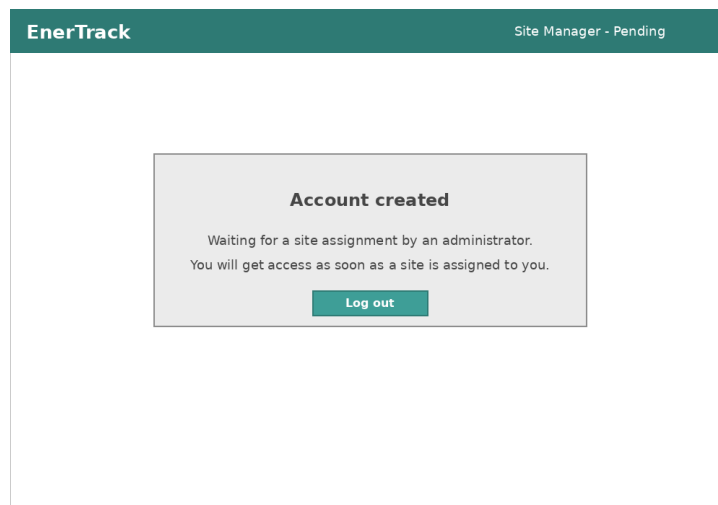
### 3.5 Middleware / Route Protection Design

RLS protects the data itself; middleware.ts adds a faster, UX-facing layer that redirects before a page loads. It is not a replacement for RLS. On each request it reads the user's role and site\_id from profiles (server-side, using the same source as the policies) and applies:

| Route         | Allowed roles                                   | If blocked, redirect to                             |
|---------------|-------------------------------------------------|-----------------------------------------------------|
| /sites (list) | Energy Manager, Direction                       | site_manager → /my-site                             |
| /sites/:id    | All (Site Manager only if it is their own site) | site_manager on another site → /my-site             |
| /my-site      | Site Manager                                    | energy_manager / direction → /sites                 |
| /consolidated | Energy Manager, Direction                       | site_manager → /my-site                             |
| /users        | Direction                                       | site_manager → /my-site;<br>energy_manager → /sites |

- Independently of the table, any Site Manager whose site\_id is null is redirected to /pending on every route until Direction assigns a site.
- The file must sit at the project root (next to app/), otherwise Next.js silently never runs it.

### Screen 7: Pending assignment (wireframe)



## 4. API Design

Sign-up, login and logout are handled directly by Supabase Auth. Password reset is out of scope for the MVP. Application routes below live under `app/api/` in `Next.js`.

### Sites

| Route                       | Method       | Access                                 |
|-----------------------------|--------------|----------------------------------------|
| <code>/api/sites</code>     | <b>GET</b>   | All (filtered by RLS)                  |
| <code>/api/sites</code>     | <b>POST</b>  | Direction                              |
| <code>/api/sites/:id</code> | <b>GET</b>   | All (if authorized by RLS)             |
| <code>/api/sites/:id</code> | <b>PATCH</b> | Direction (edit or archive via status) |

### Readings

| Route                          | Method        | Access                             |
|--------------------------------|---------------|------------------------------------|
| <code>/api/readings</code>     | <b>GET</b>    | All (filtered by site via RLS)     |
| <code>/api/readings</code>     | <b>POST</b>   | Site Manager (own site), Direction |
| <code>/api/readings/:id</code> | <b>PATCH</b>  | Site Manager (own site), Direction |
| <code>/api/readings/:id</code> | <b>DELETE</b> | Site Manager (own site), Direction |

### AI Summaries

| Route                                   | Method      | Access                                             |
|-----------------------------------------|-------------|----------------------------------------------------|
| <code>/api/ai-summary</code>            | <b>POST</b> | Site Manager (own site), Energy Manager, Direction |
| <code>/api/ai-summaries?site_id=</code> | <b>GET</b>  | All (if authorized by RLS)                         |

### Users & Roles

| Route                          | Method       | Access                                 |
|--------------------------------|--------------|----------------------------------------|
| <code>/api/users</code>        | <b>GET</b>   | Direction                              |
| <code>/api/users/:id</code>    | <b>PATCH</b> | Direction (role / site assignment)     |
| <code>/api/users/invite</code> | <b>POST</b>  | Direction (service-role key, see note) |

#### Exception to “the API is not a security boundary”

`/api/users/invite` uses the Supabase admin API with the service-role key, which bypasses RLS entirely. RLS is therefore inactive on this route, so the handler must itself verify that the caller is a signed-in Direction user before doing anything, and the key must only ever exist server-side.

Every other route is a convenience layer: even a direct or malformed request is still constrained by the RLS policies in Section 3.

## Request / response definitions

Successful responses wrap the payload in a data field. Errors share one format, and the status code says what went wrong:

```
{ "error": { "code": "FORBIDDEN", "message": "Not allowed for your role" } }
```

| Status    | code             | When                                                                                  |
|-----------|------------------|---------------------------------------------------------------------------------------|
| 200 / 201 | —                | Success (201 for a created resource)                                                  |
| 401       | UNAUTHENTICATED  | No valid session                                                                      |
| 403       | FORBIDDEN        | Signed in, but the role is not allowed (route check or RLS on a write)                |
| 404       | NOT_FOUND        | Resource does not exist, or is not visible to this user through RLS                   |
| 422       | VALIDATION_ERROR | Invalid body or constraint violation (negative value, future date, duplicate reading) |

### Example: create a reading

```
POST /api/readings
{ "site_id": "<uuid>", "energy_type": "electricity",
  "value": 4200, "date": "2026-03-01" }

201 Created
{ "data": { "id": "<uuid>", "site_id": "<uuid>",
  "energy_type": "electricity", "value": 4200,
  "date": "2026-03-01", "created_by": "<uuid>" } }

422 Unprocessable (same site, type and date already recorded)
{ "error": { "code": "VALIDATION_ERROR",
  "message": "A reading already exists for this site, type and date" } }
```

### Example: generate an AI summary

```
POST /api/ai-summary
{ "site_id": "<uuid>" }

201 Created
{ "data": { "id": "<uuid>", "site_id": "<uuid>",
  "summary_text": "Electricity use is 12% above the monthly target...",
  "created_at": "2026-03-02T09:14:00Z" } }
```

## 5. Directory Structure

Single Next.js (App Router) project covering both frontend and backend. middleware.ts sits at the project root, not inside app/.

### Pages & API routes

```

enertrack/
|-- middleware.ts           Role check + redirect (e.g. -> /pending)
|-- app/
|   |-- (auth)/
|   |   |-- login/page.tsx   Screen 1: Login
|   |   |-- signup/page.tsx
|   |-- (dashboard)/
|   |   |-- sites/
|   |   |   |-- page.tsx     Screen 2: Site list
|   |   |   |-- [id]/page.tsx Screen 4: Site detail dashboard
|   |   |-- my-site/page.tsx Screen 3: My Site (Site Manager)
|   |   |-- consolidated/page.tsx Screen 5: Consolidated dashboard
|   |   |-- users/page.tsx   Screen 6: User & role management
|   |   |-- pending/page.tsx Screen 7: Pending assignment
|   |-- api/
|   |   |-- sites/route.ts
|   |   |-- readings/route.ts
|   |   |-- ai-summary/route.ts
|   |   |-- users/route.ts
|   |-- layout.tsx

```

### Components, shared logic & database

```

|-- components/
|   |-- ui/
|   |-- ReadingForm.tsx
|   |-- ConsumptionChart.tsx
|   |-- AiSummaryPanel.tsx
|-- lib/
|   |-- supabase/
|   |   |-- client.ts        Browser-side Supabase client
|   |   |-- server.ts        Server-side Supabase client
|   |-- ai/
|   |   |-- generateSummary.ts Anthropic/OpenAI API call
|-- supabase/
|   |-- migrations/          SQL: tables, constraints, functions,
|   |                           triggers, RLS policies, seed
|-- .env.example
|-- README.md

```

- middleware.ts (root) checks the signed-in user's role and site\_id and redirects accordingly, e.g. to /pending when site\_id is null.
- lib/supabase/ separates browser and server clients, both required for RLS to apply consistently across rendering contexts.
- lib/ai/ isolates the LLM call in one place for easier maintenance and prompt iteration in Phase 3.
- supabase/migrations/ keeps every security rule (policies, triggers, functions) versioned in the repository.

## 6. Design Decisions Following Review 2

How each Review 2 point was resolved, and where in this document.

| Review point                              | Decision                                                                                                                                     | Section  |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>profiles.role can be self-modified</b> | Trigger comparing old/new row blocks role and site_id changes by non-Direction users; attacker test planned.                                 | 3.3      |
| <b>Single source of truth for role</b>    | SECURITY DEFINER helper functions used by policies, triggers and middleware; middleware queries profiles per request (no JWT claims in MVP). | 3.1, 3.5 |
| <b>Who creates the profiles row</b>       | Trigger on auth.users at sign-up. Public sign-up is primary; invite is for Energy Manager / Direction accounts.                              | 3.4      |
| <b>Invite route bypasses RLS</b>          | Handler must check the caller is Direction itself; documented as the one exception.                                                          | 4        |
| <b>middleware.ts location</b>             | Moved to project root.                                                                                                                       | 5        |
| <b>API responses and errors</b>           | Common error format, status codes 401/403/404/422, two response examples.                                                                    | 4        |
| <b>Units</b>                              | One fixed canonical unit per energy type, no free-text unit.                                                                                 | 1        |
| <b>Duplicates and invalid values</b>      | UNIQUE (site, type, date); CHECK value >= 0 and date <= today.                                                                               | 1        |
| <b>1 site = 1 Site Manager</b>            | Partial unique index on profiles.site_id.                                                                                                    | 1        |
| <b>Site archiving</b>                     | status column on sites; archive through UPDATE, no physical delete.                                                                          | 1, 2     |
| <b>Author names for Energy Manager</b>    | Energy Manager may read all profiles (read-only).                                                                                            | 2, 3.2   |
| <b>Index and Pending wireframe</b>        | Index on readings (site_id, date); Pending screen wireframe added (Screen 7).                                                                | 1, 3.5   |
| <b>Forgot password</b>                    | Out of scope for the MVP.                                                                                                                    | 4        |